



CL-2001

Data Structures

Project (Maximum 3 Group Members)

Note: Carefully read the following instructions (*Each instruction contains a weightage*)

1. Use understandable name of variables.
2. Write a code in C++ language.
3. **Submit .cpp files.**
4. Code the problem statement on MS Visual Studio C++ compiler, It is a console based project. No graphics are included.
5. Please submit a zip file in this format **Group members rollno_DS LabProject**.
6. Do not submit your project after deadline. Late and email submission is not accepted.
7. Do not copy code from any source otherwise you will be penalized with **ZERO** marks in the Project.



Semester Project

Emergency Response Dispatch System (ERDS Lite)

1. Introduction

In this project, students will implement a console-based, C++ application that simulates a centralized **Emergency Response Dispatch System**. The system must efficiently manage incoming emergency incidents and dispatch the closest available resources using fundamental data structures implemented manually.

The system will be console-based, and each resource (Incident, Vehicle) will be identified using a unique ID.

Every required component, such as incident prioritization, resource lookup, proximity search, and historical logging, must be implemented with the appropriate data structure.

This project focuses on:

- Understanding of core data structures, including **Heap** and **AVL/BST**.
- Real-world mapping of Data Structures into a logistics and prioritization system.
- Creating a modular, scalable console application.
- Hands-on practice without STL (must create your own stack, queue, heap, hash table, graph, etc.).

2. Incident and Resource Structure

2.1 Incident Data (Min-Heap Node)

Each incoming emergency call/incident is a node in the Min-Heap.

1. **Incident ID** (unique identifier)
2. **Location ID** (references a Node in the Graph)
3. **Type** (Medical, Fire, Crime)
4. **Urgency Level** (1=High, 5=Low)
5. **Status** (Pending, Dispatched, Closed)

2.2 Vehicle/Resource Data (Hash Table Value)

Each dispatch vehicle is stored in the Hash Table for $O(1)$ access.

1. **Vehicle ID** (Key for Hash Table)
2. **Type** (Ambulance, Fire Truck, Police Car)



3. **Current Location ID** (references a Node in the Graph)
4. **Status** (Available, En Route, Maintenance)

3. Incident Prioritization (Min-Heap Implementation)

The Min-Heap manages the order of incidents waiting for dispatch.

- **Structure:** Implement a **Min-Heap** using a dynamic array (or a manual array-based structure).
- **Why Min-Heap?** To always extract the incident with the lowest urgency number (highest priority) in $O(\log n)$ time.

3.1 Requirements

- Implement a **Min-Heap** from scratch (no STL).
- Constraint: You must write your own Heapify Up and Heapify Down logic.
- The Urgency Level (1-5) determines the priority.
- **Insert:** Add a new incident to the heap and perform Heapify Up.
- **ExtractMin:** Retrieve and remove the highest-priority incident, performing Heapify Down operation.

4. Resource Lookup & Status Control (Hash Table Implementation)

The Hash Table ensures quick access to vehicle status and location.

- **Structure:** Implement a **Hash Table** with an array of buckets.
- **Key:** Vehicle ID.
- **Collision Handling:** Must use **Chaining** (manual Linked List) to handle collisions.

4.1 Required Features

- **Insert:** Add a new vehicle to the Hash Table.
- **Search:** Find a vehicle by ID in $O(1)$ average time.
- **Update Status:** Change a vehicle's status (e.g., to 'En Route') and location.

5. City Map & Proximity Search (Graph & BFS)

The Graph models the city, and BFS is used for proximity-based resource allocation.

- **Structure:** Implement a **Weighted Graph** using a manual **Adjacency List** (linked-list based).

5.1 Graph Details

- Each intersection/location = **Vertex**
- Each road = **Weighted Edge** (Weight = Travel Time/Distance)

5.2 Requirements

- Represent the graph using an Adjacency List.
- Add/remove locations and roads.
- **Proximity Search:** When dispatching, use **Breadth-First Search (BFS)** to find the closest *available* vehicle (found in the Hash Table) to the incident location, based on the minimum number of "hops" (intersections).
- The BFS must use a manually implemented **Queue** for its traversal logic.

6. Historical Logging (AVL/BST)

All resolved incidents must be permanently stored for reporting and analysis.

- **Structure:** Implement an **AVL Tree** (recommended) or a **Binary Search Tree (BST)**.
- **Node Data:** Completed Incident Record (ID, Priority, Time Closed, Response Time).
- **Sorting:** The tree must be sorted by **Incident ID** or **Time Closed**.

6.1 Required Features

- **Insert:** Add a new closed incident record. If using AVL, this must include logic for calculating balance factors and performing **Rotations (LL, RR, LR, RL)**.
- **Search:** Find a historical incident by ID in $O(\log_{10} n)$ time.
- **Generate Report:** Display all records in sorted order using an In-Order Traversal.

7. Task and Command Management (Queue & Stack)

These structures manage non-critical administrative tasks and dispatcher action logs.

- **7.1 Vehicle Maintenance Tasks (Queue-Based)**

- **Structure:** Implement a **Queue** using a manual Linked List (FIFO).
- **Purpose:** Manages routine check-ups or non-urgent repairs for vehicles.
- **Features:** Enqueue Task, Dequeue (Process Next Task), View Pending Tasks.

- **7.2 Dispatcher Command History (Stack-Based)**

- **Structure:** Implement a **Stack** using a manual Linked List (LIFO).
- **Purpose:** Logs the last N critical actions (e.g., "Dispatched AMB-101").
- **Features:** Push Action, Pop (Review Last Action), Peek.



8. System & Menu Functionalities

The program must include a well-structured console menu.

8.1 Incident Dispatch Core

1. **Log New Incident** (to Min-Heap)
2. **Dispatch Next Incident** (ExtractMin from Heap → Run BFS Proximity Check → f_0 Update Hash)
3. **Resolve/Close Incident** (Update Hash Status → f_0 Insert Record into AVL/BST)

8.2 Vehicle & Map Management

1. **Add/Remove Vehicle** (Hash Table)
2. **Update Vehicle Status/Location** (Hash Table lookup)
3. **Add/Remove Map Location** (Graph Node)
4. **Add/Update Road** (Graph Weighted Edge)

8.3 Administration & Reporting

1. **View Task Queue** (Maintenance)
2. **Process Next Task** (Maintenance)
3. **Review Command History** (Stack)
4. **Generate Historical Report** (AVL In-Order Traversal)
5. **Save/Load System State** (File I/O)

9. Scalability & Modular Design

You must create modular classes for organization and future feature expansion, such as:

- IncidentMinHeap
- VehicleHashTable
- CityMapGraph
- HistoricalAVL (or HistoricalBST)
- TaskQueue
- CommandStack

Each module should be easy to extend.



10. Error Handling & Validation

System must show professional errors, for example:

- "Incident ID already exists."
- "Vehicle AMB-101 not found."
- "No available vehicles of type [Medical] found within 3 hops."
- "Heap is empty. No pending incidents."

IMP Note:

Data Persistence (File Handling)

The final application must implement **File Input/Output (I/O)** using standard C++ file streams (<fstream>) to ensure the system's state is preserved between sessions. You must implement both **Save** and **Load** functionalities. Upon program exit, the system must **Save** the complete content of the primary data structures, including all pending incidents in the **Min-Heap**, all vehicles and their status in the **Hash Table**, and all historical records in the **AVL/BST**. Conversely, when the program starts, it must automatically attempt to **Load** this saved state to seamlessly restore the previous dispatch environment, including all node data and connections for the Graph. This process demonstrates a robust, real-world application's ability to maintain data integrity.

Best of luck 😊